

KupeFDX-hi-asr: A Streaming Hindi Speech Recognition and Floor-Control System Fusing a Self-Supervised Omnilingual Encoder with the Nandi-Mini Small Language Model

A Method Proposal

FrontiersMind / Kupe FDX — Research

September 2026

Abstract

We propose **KupeFDX-hi-asr**, a single streaming system for Hindi voice agents that performs three tasks jointly: (i) accurate transcription, (ii) live domain-terminology correction, and (iii) almost-full-duplex *floor control* — deciding *when to stay silent* and, rarely, when to emit a backchannel, a thinking-sound, or an end-of-speech signal. The system fuses the **self-supervised (SSL) omniASR_W2V** (wav2vec 2.0) encoder from Meta’s Omnilingual ASR release [1] — using *only* its SSL weights, not the CTC-finetuned head — with the Hindi-native **Nandi-Mini-150M** small language model [2]. The architecture is encoder-agnostic, so Fast-Conformer/IndicConformer remains a one-flag fallback if Hindi WER stalls (Section 5). **Nandi produces the final transcript**; our own connectionist-temporal-classification (CTC) head [4] is only a helper — it fine-tunes the encoder on Hindi and supplies end-of-speech timing, never the authoritative transcript, and none of Meta’s CTC weights are loaded. A projector maps continuous encoder features into Nandi’s embedding space, and an optional k -means quantizer can add discrete “audio tokens” to Nandi’s vocabulary. Crucially, a dedicated per-chunk floor-control head is trained so that its default, confident output is *NOTHING*: the model stays quiet on the overwhelming majority of chunks and emits a signal only when it clearly fires. This document specifies the architecture, the mathematics of every component and loss, the five-phase training schedule, the audio-aware data-generation procedure with its scenario distribution and JSON formats, and the streaming input/output contract. Per the request, results and conclusions are deliberately excluded — this is a method plan. All quantitative details below are drawn directly from the reference implementation.

1 Introduction and Motivation

Production Hindi voice pipelines chain separate models for transcription, correction and turn-taking. They sound robotic (dead silence while “thinking”), miss domain terms, and fire turn-boundary signals at the wrong moments. We instead fuse two open-weight foundations into one proprietary system. The encoder is the *raw self-supervised* variant of Omnilingual ASR: it emits continuous contextual vectors and, having no baked-in output vocabulary, sidesteps any tokenizer-mismatch problem [1]. The decoder is Nandi-Mini-150M, trained from scratch on 525 B tokens across English and ten Indic languages with native Devanagari coverage [2], which supplies a strong

Hindi prior for correction and for generating natural backchannel words. The scope here is Hindi; the design is deliberately built as a floor controller from the start so it validates on one language before scaling to the remaining Indic languages plus English.

2 Plain-Language Overview (for a new engineer)

If you are new to speech ML, here is the whole project in everyday terms; every term is defined again in detail later.

- **The goal.** Build one model that, while a person speaks Hindi, (1) writes down what they say, (2) fixes special words (medical/technical terms), and (3) decides *when to react* — say a quick “हैं” (backchannel), make a thinking sound “हम्म”, or detect that the person finished talking (end-of-speech). Most of the time it should do nothing except keep writing.
- **Two ready-made models we glue together.** An encoder (omniASR_W2V) that turns sound into a list of number-vectors --- think of it as “ears”. And a small language model (Nandi-Mini-150M) that is good at Hindi text --- think of it as the “mouth/brain”. The encoder does not know any words by itself; the language model does.
- **The bridge.** A tiny neural layer called a projector translates the encoder's number-vectors into the “language” the SLM understands, and we feed them in as if they were words. **Nandi (the SLM) writes the final transcript** --- not the encoder.
- **The CTC head is a helper, not the writer.** We attach a simple CTC head (one linear layer) to the encoder, but only to (a) fine-tune the encoder on Hindi in Stage A and (b) detect where the silences/turn-ends are. It never produces the real transcript --- that is always Nandi's job. (We also never load Meta's own CTC weights.)
- **The flags.** We add five special words to the SLM's vocabulary: <BC> (backchannel), <THINK>, <EOS_SPEECH> (end of speech), <SILENCE>, and “nothing”. A small floor-control head predicts one of these for every short chunk of audio; almost always the answer is “nothing”.
- **The data.** Normal speech datasets give us audio + transcript. They do not tell us where the flags go. So we use an LLM (OpenAI gpt-5.6-luna) as a data factory: we measure the pauses in each audio clip and ask the LLM to place the flags sensibly, producing labelled examples in JSON.
- **Training in three stages.** We do not train everything at once. Stage A --- fine-tune the ears (encoder) on Hindi. Stage B --- teach Nandi to read the (now Hindi-tuned) encoder's audio and write the transcript. Stage C --- teach the combined model the flags and word-correction. (Under the hood these are five resumable training runs; Table 2 shows the data each uses.)
- **How you run it.** Every step is one command (commands.md); a smoke test (bash run_smoke.sh) runs the entire pipeline on a laptop in seconds with tiny stand-in models, so you can verify the wiring before spending GPU money. On the H100 you flip one config flag (backend: real) and the same code loads the real models.

3 Short Literature Review

Self-supervised encoders. wav2vec 2.0 learns speech representations by contrastive prediction over masked latent frames [3]; Omnilingual ASR scales this encoder to 7 B parameters and covers 1,600+ languages, reporting character error rate (CER) below 10 for 78% of them [1]. Meta releases two decoders on top of the *same continuous* encoder: a CTC head that is “a single linear

layer...optimized with a CTC loss”, and an “LLM-ASR” transformer decoder that attains their best accuracy [1]. Notably, *neither uses k -means or discrete units*; both consume continuous features. **Discrete units** are a separate lineage: HuBERT derives targets by k -means clustering of features [5], which underpins textless speech-LM work but discards acoustic detail. **Speech-to-LLM** systems such as Whisper’s encoder-decoder [6], Qwen-Audio [7, 8] and SALMONN [9] feed *continuous* audio features into a language model via a projector, which is the regime we adopt. **Turn-taking** has been modelled by Voice Activity Projection, which predicts upcoming speech activity to manage backchannels and turn shifts [10]. **Indic ASR** resources — IndicWav2Vec [11] and the 6,400 h Shrutilipi corpus [12] — make thousands of hours of Hindi available. Our contribution is to *fuse* a raw Omnilingual encoder with a Hindi-native SLM into one streaming system that transcribes, corrects, and controls the floor, with an explicit stability guarantee against over-firing.

4 System Overview

Figure 1 shows the single forward path. Audio is denoised and passed through the causal SSL encoder. Its continuous features fan out to (a) the CTC head, (b) the projector that produces 832-dimensional soft prompts for Nandi, (c) an optional k -means quantizer producing discrete audio-code tokens, and (d) the per-chunk floor-control head. Nandi autoregressively decodes the corrected transcript with inline control signals, conditioned on a domain tag and rolling conversation context.

Figure 1. KupeFDX-hi-asr architecture (streaming, single forward path)

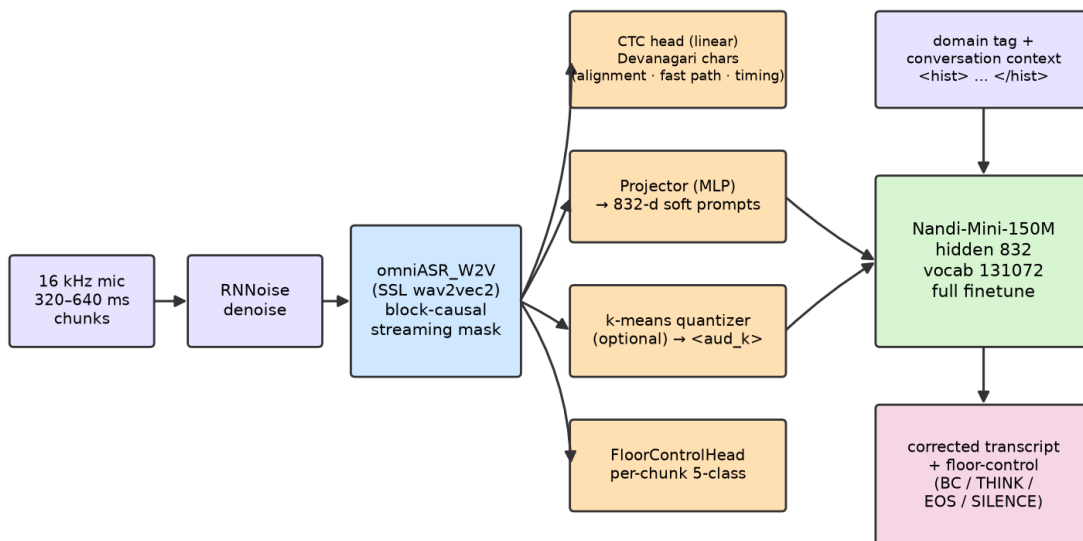


Figure 1: KupeFDX-hi-asr architecture. The encoder is used in a block-causal (streaming) mode; the CTC head is retained for the whole project.

5 Encoder and Streaming (Block-Causal) Masking

The encoder `Enc` maps a waveform x (16 kHz mono) to features $H = \text{Enc}(x) \in \mathbb{R}^{T \times D}$ at a frame rate of 50 fps (hop 320 samples). To make the *validated* word error rate a genuine streaming number rather than an offline one we later regress, we impose a block-causal attention mask from the first phase. With chunk size C frames and left context of L chunks, define the chunk index $c(i) = \lfloor i/C \rfloor$; the additive mask is

$$M_{ij} = \begin{cases} 0 & \text{if } c(i) - L \leq c(j) \leq c(i), \\ -\infty & \text{otherwise.} \end{cases} \quad (1)$$

Thus frame i attends only within its own chunk and the L preceding chunks, never to the right. In the reference configuration $C=32$ frames (≈ 640 ms) and $L=2$. A unit test verifies no future leakage: perturbing the last ~ 200 ms of audio leaves earlier frame outputs unchanged. The encoder wrapper exposes a stable interface $(H, \ell) = \text{Enc.features}(x)$ with per-clip frame lengths ℓ , so the raw Omnilingual backbone (`backend: real`) and a small stand-in used for testing are interchangeable.

Which encoder, and the SSL-only rule (important). We use the **Omnilingual omniASR_W2V encoder**, and we use **only its self-supervised (SSL) wav2vec 2.0 weights**. We do *not* load Meta’s CTC-finetuned checkpoint (`omniASR-CTC-*`) or any of its decoder/head weights — we attach and train *our own* CTC head (Section 6) from scratch in Phase 1. The SSL encoder has no output vocabulary baked in, which is exactly why it composes cleanly with Nandi and our fresh CTC head (no tokenizer mismatch). The loader (`encoders.py`) refuses a `*-CTC-*` id to make this rule impossible to violate by accident.

Honest risk note. Two independent 2026 real-world Hindi benchmarks measure the *full* Omnilingual ASR system as weaker than FastConformer/IndicConformer on Hindi — Vaani [15]: 26.4 WER for `omniASR_LLM_1B` vs 10.6/14.2 for FastConformer/IndicConformer; Voice-of-India [16]: 13.7–14.9 vs 8.2. Those numbers are for Meta’s *off-the-shelf* system, not for our recipe (a Hindi-fine-tuned SSL encoder + our CTC head + Nandi correction), so they are a documented risk, not a verdict on our design. Because the architecture is encoder-agnostic (the projector adapts any encoder width), **FastConformer/IndicConformer is a one-line fallback** (`encoder_type: fastconformer`) if Phase-3 Hindi WER stalls above target. Everything else in this proposal is unchanged by the swap. (See `ENCODER_DECISION.md` for the full measured comparison.)

How much audio reaches the SLM, and how (Figure 2). A 640 ms streaming chunk of 16 kHz mono audio is 10,240 samples. The `omniASR_W2V` SSL encoder turns it into 32 frames (20 ms/frame, 50 fps), each a *continuous* vector. The projector maps each frame to one 832-d soft prompt, so Nandi receives 32 continuous 832-d vectors per chunk (FastConformer would give 8), wrapped in `<audio> ... </audio>` and fed as `inputs_embeds`. Nothing is discretized. A rolling buffer plus the block-causal mask (left context 2 chunks) makes this a true streaming forward.

6 The CTC Head: Continuous Features, No k -means

Following Omnilingual’s CTC recipe [1], the head is a single linear map from the D -dimensional continuous features to a Devanagari character vocabulary of size V_c (index 0 is the CTC blank), followed by a log-softmax. For target character sequence y and input length T , the CTC loss

Figure 7. Audio $\rightarrow$ SLM: chunk sizes and how much is passed (continuous, per chunk)

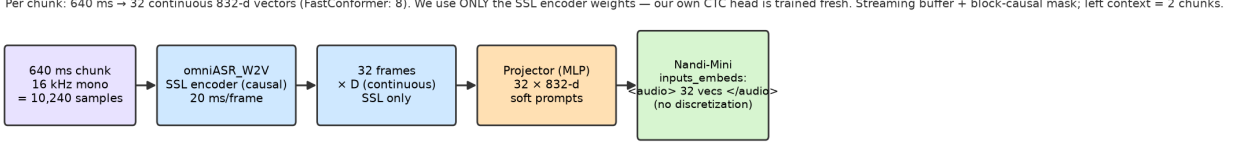


Figure 2: Per-chunk audio $\rightarrow$ SLM: 640 ms $\rightarrow$ 32 continuous 832-d soft prompts (omniASR_W2V SSL; FastConformer would give 8).

marginalizes over all frame-level alignments π that collapse (remove repeats, then blanks) to y :

$$\mathcal{L}_{\text{CTC}} = -\log \sum_{\pi \in \mathcal{B}^{-1}(y)} \prod_{t=1}^T p(\pi_t | H). \quad (2)$$

The CTC head is a helper, not the transcript producer. Its purpose is (a) to be the objective that fine-tunes the Omni SSL encoder on Hindi in Stage A, (b) a monotonic-alignment anchor that speeds Nandi’s convergence and curbs hallucination, and (c) a free end-of-speech/silence *timing* signal — a long run of CTC-blank frames at the tail marks a probable turn end. **The final transcript always comes from Nandi** (`model.generate`); the greedy CTC string is at most a throwaway low-latency draft (off by default in streaming). This also answers whether k -means is needed: it is not — the self-supervised continuous encoder plus a linear CTC head is enough to adapt the encoder, with no discretization, exactly as in Meta’s release. We never load Meta’s CTC weights; our head is trained from scratch.

7 Projector and Optional Discrete Audio Tokens

The projector (a two-layer MLP with LayerNorm and a GELU nonlinearity) maps each continuous frame to Nandi’s input width $E=832$, and owns two learned marker vectors $\langle \text{audio} \rangle, \langle / \text{audio} \rangle$. These continuous soft prompts are the primary way Nandi “hears” the encoder — the same continuous-feature regime used by SALMONN and Qwen-Audio [9, 7], and by Omnilingual’s own LLM-ASR decoder [1], and the recommended default.

To additionally give Nandi *discrete* audio tokens (so that acoustic units live in the same embedding table as text), we optionally quantize features with k -means. Fitting solves Lloyd’s objective over a feature sample $\{f_n\}$ with K centroids $\{\mu_k\}$,

$$\min_{\{\mu_k\}} \sum_n \|f_n - \mu_{a(n)}\|_2^2, \quad a(n) = \arg \min_k \|f_n - \mu_k\|_2^2, \quad (3)$$

and each frame is encoded to its nearest centroid index, surfaced as a vocabulary token $\langle \text{aud_k} \rangle$. Because discretization discards acoustic detail, we treat it as an additive ablation ($K=2048$ when enabled, 0 by default); residual vector quantization is a natural stronger alternative if discrete tokens prove useful. The recommendation is therefore: *use continuous soft prompts as the primary signal; add k -means audio tokens only if an ablation shows they help.*

8 The Nandi Decoder, Token Extension, and Tokenizer Fertility

Nandi-Mini-150M is a decoder-only transformer with hidden size 832, a BPE vocabulary of 131,072 with native Devanagari, 16 layers applied with layer-sharing ($\times 2$), grouped-query attention (16 query / 4 key-value heads), SwiGLU, RMSNorm, rotary position embeddings ($\theta=10^5$), a 2048-token context, tied input/output embeddings, and a *factorized* embedding of rank 196 [2]. Audio enters as `inputs_embeds`, so Nandi’s text vocabulary is never resized for text; we extend it only by a small set of new tokens: the audio and history markers, the floor-control flags, and (optionally) the K audio-code tokens. Extension appends rows to the factorized embedding table $W \in \mathbb{R}^{V \times 196}$ and, because the head is tied, both input and output sides grow consistently; each new row is initialized to the mean of existing rows plus small noise. This is the single riskiest piece of the fusion and is covered by a unit test that runs on a faithful structural mirror (and, where reachable, on the real model).

Tokenizer fertility. Fertility is the mean number of subword tokens per word,

$$\mathcal{F} = \frac{\text{\#subword tokens}}{\text{\#words}}, \quad (4)$$

measured on a representative Hindi set by `scripts/08_tokenizer_fertility.py`. Lower fertility means shorter target sequences, faster decoding, and more of Nandi’s 2048-token budget left for audio soft prompts and conversation context. A native-Devanagari BPE such as Nandi’s is expected to fragment Hindi far less than a character-level baseline (whose fertility upper-bounds at the characters-per-word count); the script reports the measured value together with characters-per-token and bytes-per-token.

Sequence layout. Figure 3 shows the sequence assembled per example: an optional context prefix, then `bos`, the audio markers wrapping the projected frames (and any audio-code tokens), then the supervised target. The prefix is masked out of the loss (label -100).

Figure 2. Sequence fed to Nandi as `inputs_embeds` (text vocab reused, not resized)

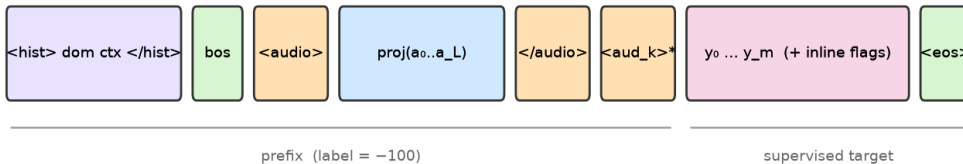


Figure 3: Token sequence fed to Nandi as continuous embeddings; only the target span is supervised.

9 Floor Control: Inline Flags and a Stable Per-Chunk Head

Four mutually exclusive signals are defined with non-overlapping semantics: `<EOS_SPEECH>` (the user turn is genuinely finished — semantic completeness *and* a real trailing pause), `<BC>` (a short listener acknowledgment *while the user is still speaking*, carrying a surface word such as `हाँ`), `<THINK>` (a filler while the system composes a reply, only after `<EOS_SPEECH>`, e.g. `हम्म`), and `<SILENCE>` (a sustained low-speech stretch that is not a turn boundary). The absence of any flag denotes nothing happens.

Inline representation. In the autoregressive target, a flag is written only at the moment it occurs; there is no per-frame <NOP>. A backchannel renders as “<BC> हँ”; end-of-speech as “<EOS_SPEECH>”; an ordinary or mid-utterance pause renders nothing. This absence is the core defence against over-firing: a mid-sentence pause is present in the audio features but carries no flag in the target, teaching restraint. During floor-control training the inline flag positions are up-weighted so that rare signals are learnable without the no-flag majority drowning them: with the set of flag positions $\mathcal{S}$,

$$\mathcal{L}_{\text{fc}}^{\text{inline}} = \frac{1}{|\mathcal{S}|} \sum_{t \in \mathcal{S}} -\log p(y_t | y_{<t}). \quad (5)$$

Per-chunk head (the streaming stability layer). A streaming agent is queried every chunk, and the answer must be no most of the time. A dedicated head classifies each chunk into one of five classes $\{\text{NOTHING}, \text{<BC>}, \text{<THINK>}, \text{<EOS_SPEECH>}, \text{<SILENCE>}\}$. The <BC> surface may be a plain acknowledgment (हाँ, अच्छा, जी) or an emotional expression --- laughter हाहाहा, a sigh उफ़, surprise ओह/अरे --- generated by Nandi when the context warrants it. Encoder frames in chunk c are mean-pooled to $\bar{h}_c$ and scored by a two-layer MLP; training minimizes a class-weighted cross-entropy

$$\mathcal{L}_{\text{fcc}} = - \sum_c w_{y_c} \log p(y_c | \bar{h}_c), \quad w = (0.3, 2, 2, 2, 2), \quad (6)$$

where labels come from the timeline: every ordinary ASR chunk is a NOTHING label, so NOTHING dominates the data and the head learns to stay quiet; the weights up-weight the rare classes only enough to keep them learnable. This head already receives the NOTHING baseline in Phase 3 (weight 0.2 on pure-ASR chunks) before Phase 4 adds the rare events.

Inference guard. At run time a decider adds two hard guards on top of the softmax p at chunk c (Figure 4). It emits class $k^* = \arg \max_k p_k$ only if

$$p_{k^*} > \tau_{k^*} \quad \text{and} \quad p_{k^*} > p_{\text{NOTHING}}, \quad (7)$$

with thresholds $\tau = 0.5$ for <BC>/<THINK>/<EOS_SPEECH> and $\tau = 0.6$ for <SILENCE>; additionally <BC>/<THINK> require a minimum gap of 3 chunks since the last fire (anti-chatter), and <EOS_SPEECH> latches until the turn resets. Thus even if logits wobble the system cannot fire on every chunk --- the quiet default holds by construction.

Controllable at inference (the temperature analog). Firing is a dial, not a fixed policy. A `StreamControls` object exposes, per signal, a probability threshold, a logit bias (eagerness: + fires more, − suppresses), a minimum chunk gap, and an on/off switch, plus a global softmax temperature τ_{soft} scaling $z \rightarrow z/\tau_{\text{soft}}$ before the decision. An operator can thus make the agent backchannel more (raise $\text{bias}[\text{<BC>}]$), commit to end-of-turn more conservatively (raise $\tau_{\text{<EOS_SPEECH>}}$), or disable thinking-sounds --- all at inference, without retraining. The word is decoupled from the decision: the head decides when; Nandi generates the surface (an acknowledgment or an emotional expression) after the flag in the autoregressive pass.

10 Training Objective

The total loss combines four terms with phase-dependent weights:

$$\mathcal{L} = \lambda_{\text{lm}} \mathcal{L}_{\text{lm}} + \lambda_{\text{ctc}} \mathcal{L}_{\text{CTC}} + \lambda_{\text{fc}} \mathcal{L}_{\text{fc}}^{\text{inline}} + \lambda_{\text{fcc}} \mathcal{L}_{\text{fcc}}, \quad (8)$$

Figure 3. Streaming decision — controllable, and quiet by default

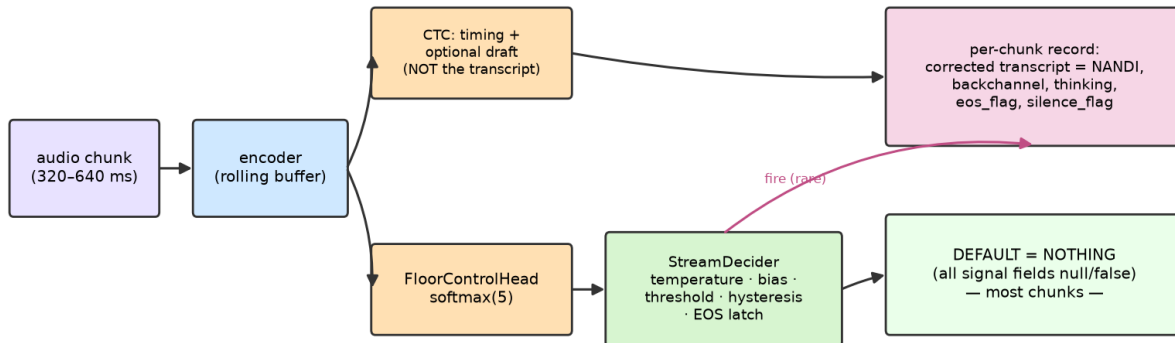


Figure 4: Per-chunk streaming decision. Most chunks return NOTHING (all signal fields null/false); a signal is emitted only when it clearly fires.

where $\mathcal{L}_{lm} = -\sum_t \log p(y_t | y_{<t}, \text{prefix})$ is the standard next-token cross-entropy over the supervised target span. Optimization uses AdamW with a cosine schedule and linear warmup, bf16 on the H100, gradient clipping, and a low-learning-rate parameter group for the encoder when it is unfrozen. Training is fully resumable: a checkpoint bundles model, optimizer, scheduler, global step and RNG state.

11 Training: Three Stages (Five Runs)

Training is organised as three stages, implemented as five resumable runs (Figure 5). **Stage A (run 1)** fine-tunes the Omni SSL encoder on Hindi using our CTC head as the objective --- this is the “teach the encoder more Hindi” step, and the CTC head is only a training tool here, not the final transcript. **Stage B (runs 2--3)** teaches Nandi to transcribe from the adapted encoder’s audio (the projector aligns the audio to Nandi’s space, then everything is fine-tuned jointly; run 3 is the main $< 5\%$ WER gate, and Nandi produces the transcript). **Stage C (runs 4--5)** adds the floor-control signals and domain-term correction on the combined model. Each run sets which sub-modules train and the loss weights, following the schedule in the implementation. The mental model: Phases 1--3 teach transcription and how to

Phase	Trains	Frozen	Loss weights ($\lambda_{lm}, \lambda_{ctc}, \lambda_{fc}, \lambda_{fcc}$)
1 CTC	encoder, CTC head	—	(0, 1, 0, 0)
2 Align	projector, Nandi	encoder, CTC	(1, 0.3, 0, 0)
3 Joint	all (encoder low-LR)	—	(1, 0.3, 0, 0.2)
4 Floor-control	projector, Nandi, FC head	encoder, CTC	(1, 0.1, 3, 1)
5 Domain	Nandi	encoder, projector, CTC	(1, 0.1, 1, 0)

Table 1: Phase schedule. Phase 3 is the main $< 5\%$ streaming-WER gate; it also pre-trains the per-chunk head on the NOTHING baseline before Phase 4 introduces rare events. Nandi is *fully* fine-tuned (single H100).

“understand” the encoder’s audio (continuous soft prompts, and optionally discrete audio tokens); Phase 4 post-trains the flags with transcription context; Phase 5 adds domain-term correction. Phases 4--5 always mix in ASR data to prevent forgetting.

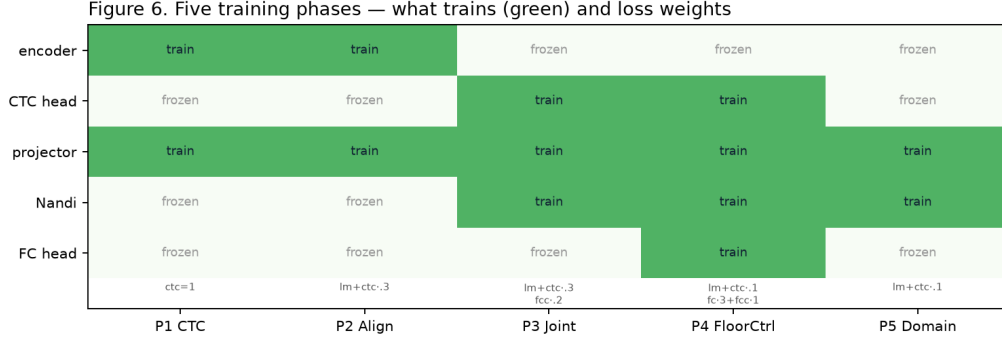


Figure 5: What trains (green) versus frozen per phase, with the loss weights.

What data goes in and comes out of each run. Table 2 is the single most useful reference for an implementer: for every run it names the stage, the exact input, the supervised target (what the model is trained to output), and the data file used.

Stg	Run	Input (per example)	Target / output (super-vised)	Data used
A	1 CTC	raw audio waveform	Devanagari <i>characters</i> (CTC) — adapts the encoder, not the final transcript	ASR ($\approx 2,500$ h)
B	2 Align	audio (cached features) + that clip’s transcript	transcript <i>tokens</i> (Nandi decodes them)	same ASR data
B	3 Joint	raw audio	transcript tokens (+ CTC aux) — Nandi is the transcript	ASR 2,500 h + domain 400 h
C	4 Floor	audio + context + domain tag	target_sequence = transcript with <i>inline flags</i> ; + per-chunk flag label (mostly “nothing”)	fc.jsonl (≈ 300 h) + mixed ASR
C	5 Domain	audio + context + domain tag	<i>corrected</i> transcript (domain terms fixed)	domain.jsonl (≈ 400 h)

Table 2: Per-phase training data: input, supervised target, and source file. In every phase the audio is the input; only the *target* and the trainable modules change. Phase 1 targets characters (CTC); Phases 2–5 target Nandi tokens; Phase 4 adds inline flags and the per-chunk floor-control label; Phase 5 changes the target from the raw to the corrected transcript.

Expectations (what “done” looks like, and the risks). This is a proposal, so no results are claimed; the targets we hold ourselves to are: Phase 1 encoder greedy-CTC WER $< \sim 9\%$; Phase 3 the main gate --- **streaming AR WER** $< 5\%$ on a clean blind test (with $\approx 3,200$ h of quality-filtered data); Phase 4 floor-control $F_1 > 0.8$ per signal with a false-fire rate $< 2\%$ on no-flag examples; Phase 5 higher domain-term accuracy with no WER regression. Realistic caveats an implementer should expect: (i) data quality dominates volume past ~ 1.5 k h --- noisy 5 k h loses to clean 2.5 k h; (ii) if Phase-3 WER stalls at 6--7%, the lever is more clean data / harder filtering, not more epochs; (iii) the Omni SSL encoder is a bet

(see the honest risk note in Section 5) --- if Hindi WER will not drop, switch the encoder to FastConformer with one config flag; (iv) the floor-control model must see far more “nothing” than signals or it will over-fire --- this is enforced by the data distribution and the inference guards.

12 Data: Hours, Sources, and Generation

Budget. We target $\approx 3,200$ curated hours (Figure 6): 2,500 h core clean ASR, 400 h domain packs (medical / technical / support / general), and 300 h floor-control. Because the SSL backbone is already massively multilingually pretrained and Nandi supplies a strong decoder prior, quality-filtered data at this scale is the realistic path to $< 5\%$ streaming Hindi WER (data quality dominates volume past ~ 1.5 k h). Public sources include Shrutilipi [12], IndicVoices, Kathbath, Vaani, Spring-INX, MUCS, Common Voice, and FLEURS; the cleanest ~ 2.5 k h are selected by a quality filter. A held-out dev set (15–20 h) and blind multi-domain test sets (≈ 25 h) are frozen at split time (speaker-disjoint, domain-stratified).

Figure 5. Training-data budget $\approx 3,200$ h (target for $< 5\%$ streaming WER)

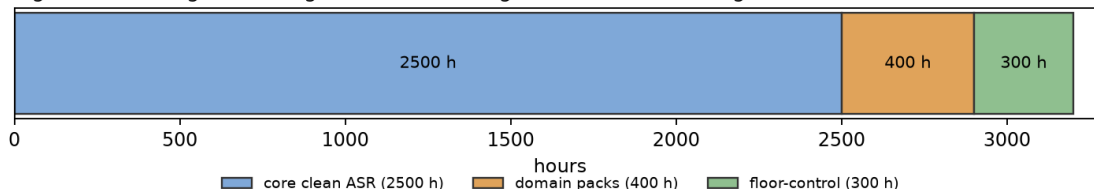


Figure 6: Training-data budget by role.

Floor-control data generation (audio-aware). Ordinary ASR corpora do not label when to fire a signal, so we synthesize that supervision. Each real clip’s waveform is read by an energy-based VAD probe that extracts pauses, durations, leading/trailing silence and speech rate, serialized to a compact textual “audio card”. An LLM agent (OpenAI-compatible endpoint; the configured model is `gpt-5.6-luna`) receives a batch of ~ 5 audio cards and produces ~ 22 structured rows per request, grounding every row in the given pauses. Generation runs with a concurrency of 10 in-flight requests and a progress bar, is resumable per batch, and every row is schema-validated (which rejects any placement that contradicts the transcript/context --- e.g. `<THINK>` without a preceding `<EOS_SPEECH>`, or `<BC>` before any speech) and re-rendered. The configured generation model is `gpt-5.6-luna`. Figure 7 shows the enforced scenario distribution, which is config-driven (`fc.distribution`): backchannel and emotional expression together are $\approx 26\%$ (humans backchannel often), while $\approx 45\%$ of rows still carry no flag at all (nothing-happens, mid-sentence pause, false-trigger, barge-in) --- the point is to fire appropriately, not rarely.

JSON format. Each row is one audio-grounded example. The trainer reads `audio` and `target_sequence`; the rest is analysis/provenance so the data is auditable and re-renderable. Fields: `id`, `lang`, `domain`, `scenario`, `audio`, `transcript` (plain --- the CTC target and WER reference), `audio_features` (`duration_s`, `pauses[]`, `trailing_silence_s`, `speech_fraction`, `mean_energy_db`), `context` (0--3 prior turns), `timeline` (speech/pause segments with optional flag and surface word), and `target_sequence` (rendered, with inline flags). The renderer maps a speech segment to its text, a `<BC>/<THINK>` pause to “`<flag> surface`”, a `<EOS_SPEECH>/<SILENCE>` pause to “`<flag>`”, and an unflagged pause to nothing.

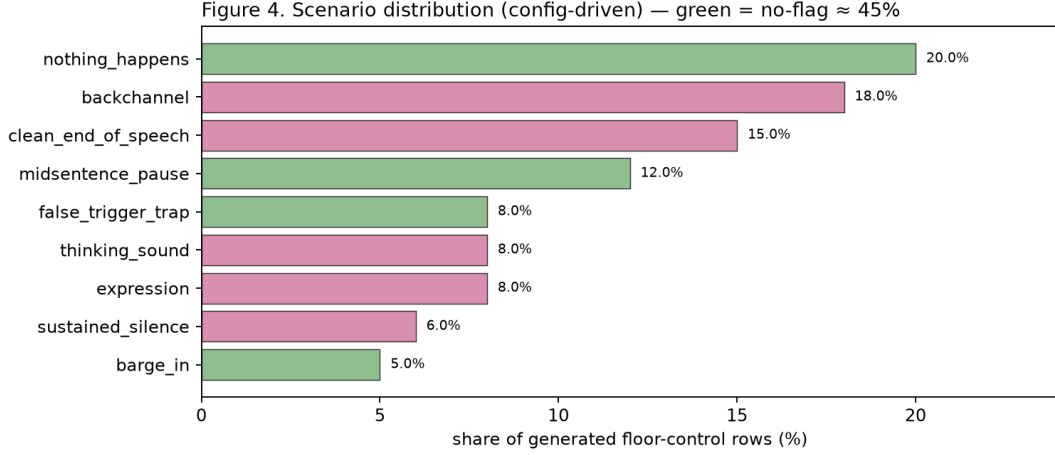


Figure 7: Scenario distribution enforced by the generator (green = no-flag).

Scenario samples (generated). Table 3 lists the nine scenarios with the rendered target from real generated rows; note the mix of plain and emotional backchannels and how context justifies each signal.

Scenario	Context (prior turn)	Rendered <code>target_sequence</code>
nothing_happens	—	मुझे दवा चाहिए
clean_end_of_speech	Agent: आपकी क्या समस्या है?	धन्यवाद आपका दिन शुभ हो <EOS_SPEECH>
midsentence_pause (trap)	Agent: हाँ जी बोलिए	नमस्ते आप कैसे हैं (pause, <i>no flag</i>)
backchannel	Agent: अपनी बात बताइए	मुझे <BC> जी दवा चाहिए
expression (emo- tional)	Agent: फिर क्या हुआ, बताइए!	नमस्ते आप <BC> आह कैसे हैं
thinking_sound	Agent: ठीक है, बताइए	मुझे दवा चाहिए <EOS_SPEECH> <THINK> हम्म
sustained_silence	—	नमस्ते आप कैसे हैं <SILENCE>
false_trigger_trap	Agent: कृपया बताइए	उम्म नमस्ते आप कैसे हैं (<i>no flag</i>)
barge_in	—	नमस्ते आप कैसे हैं (<i>no flag</i>)

Table 3: One generated row per scenario (Devanagari targets). Backchannel and expression carry emotional/plain surfaces; four of nine scenarios carry no flag by design.

Domain-correction record (Phase 5). A separate record type supervises terminology correction. Given `omni_raw_transcript` (what the acoustics literally say), `context` (prior turns) and a `domain` tag, the target is a `corrected_transcript`, with `correction_spans` listing each fix (`raw`→`fixed`, `reason`) and `chunk_boundaries_ms` for alignment. The record maps cleanly onto the training row: `text` becomes the raw transcript (the CTC / raw-acoustic target), `target_sequence` becomes the corrected transcript (Nandi’s target), and `context+domain` form the decoder prefix. The split is principled: CTC learns what the audio literally says; Nandi learns to correct it using domain and context.

How the agent knows the pauses (measured, not assumed). The agent never guesses timing. `fcgen.audio_probe` reads the waveform and runs an energy-based VAD (framed RMS in dB with a silence threshold and a minimum-pause length) to extract each internal pause’s [start, end, duration],

plus clip duration, leading/trailing silence, speech fraction, and mean energy. These measured values are serialized into a compact “audio card” that the agent must ground every row in --- it may only place flags at pause times the audio actually contains. Solid audio evidence in, not assumptions.

Agent input, output, and the static/generated split (Figure 8). The agent's input is the audio card plus the clip's real Devanagari transcript. What is **static** (from real data): the audio and the transcript. What is **generated** (by the agent): the scenario, the 0--3-turn context, and the flag placements with their surface words (the timeline). The transcript is kept verbatim --- not rewritten --- which both preserves Devanagari fidelity and saves output tokens. A merge+validate step renders the `target_sequence` (transcript with inline flags) and rejects any row that violates schema, semantic ordering, or Devanagari purity. The collator then builds the model's `inputs_embeds` sequence and the per-chunk floor-control labels (mostly NOTHING).

Figure 8. Agent input → output → merge → what the model trains on

STATIC (real data): audio + Devanagari transcript. GENERATED (agent): flags · surfaces · context. Token-saving: transcript is NOT re-emitted.

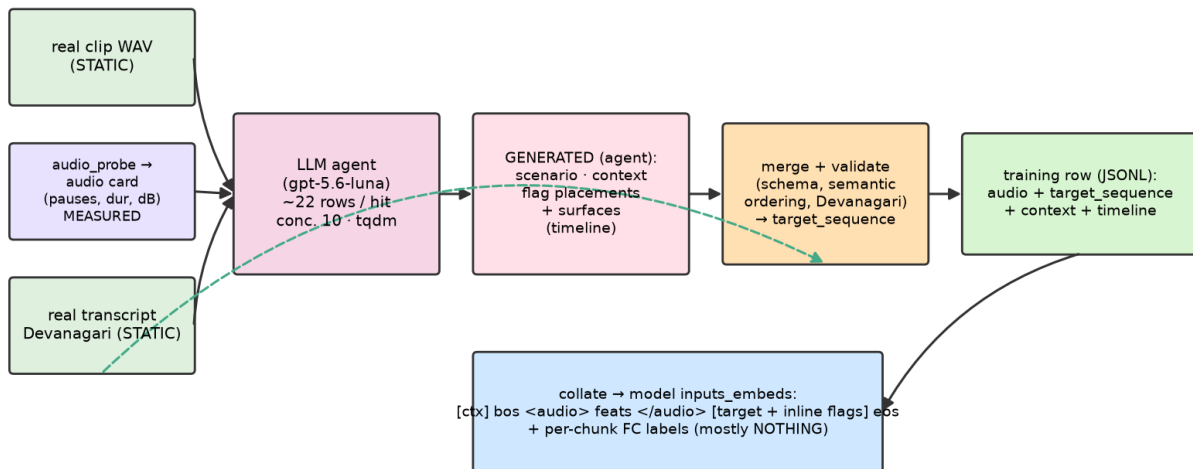


Figure 8: Agent input → generated annotations → merge/validate → training row → collated model input. Green = static (real), pink = agent-generated.

Token consumption (OpenAI gpt-5.6-luna). Batching 22 rows per call amortizes the fixed prompt (rules + 5 audio cards, ≈ 1.1 k input tokens) across the batch; with the transcript kept static, each row emits ≈ 90 output tokens (scenario, context, timeline), so a call is ≈ 1.1 k in + ≈ 2.0 k out. Table 4 gives totals; keeping the transcript static saves ≈ 20 -- 25% of output tokens versus re-emitting it.

Target rows	LLM calls	input tokens	output tokens
100k floor-control	$\approx 4,550$	≈ 5.0 M	≈ 9.1 M
200k floor-control	$\approx 9,100$	≈ 10.0 M	≈ 18.2 M
40k domain-correction	$\approx 6,700$	≈ 4.7 M	≈ 4.8 M

Table 4: Approximate OpenAI token consumption for data generation (cost = tokens $\times$ per-token price). Concurrency 10 with per-batch resume; a crash never re-spends done batches.

Strict data-quality gate. Before any training run, `scripts/09_data_quality.py` audits a manifest and gates on hard thresholds: schema + semantic-ordering validity, the per-flag distribution and no-flag ratio, per-scenario and per-domain counts, Devanagari purity (markup and genuine English terms stripped, then the Devanagari-letter ratio must clear 0.90 --- this rejects romanized Hindi such as “kaise ho”), audio existence, duration/pause sanity, and duplicates. It exits non-zero on failure so it can block a run in CI.

13 Streaming Input/Output Contract

Input: 16 kHz mono chunks of 320--640 ms, plus chat context (prior turns and a domain tag) supplied to the decoder as a `<hist> domain : turns </hist>` prefix. **Output:** one record per chunk with fields `chunk_id, timestamp_ms, raw_ctc_transcript, corrected_transcript, backchannel, thinking_sound, eos_flag, silence_flag`. On an ordinary chunk all four signal fields are null/false and only the transcript advances. The authoritative transcript is `corrected_transcript`, produced by **Nandi** (refreshed at end-of-turn and every few chunks so it is stable, not flickering). `raw_ctc_transcript` is an optional, non-authoritative low-latency draft and is off by default --- the CTC head is used at inference only for end-of-speech/silence timing, not to transcribe. When does a response come? Only when the per-chunk head's top class clearly beats NOTHING and passes the decider's threshold and hysteresis: a backchannel mid-turn, a thinking-sound post-turn, an end-of-speech latch at turn end, or a silence flag on a sustained gap --- otherwise the system is silent and simply keeps transcribing.

14 Low-Latency Design (target: end-of-turn < 100 ms)

Because the models are small (≈ 300 M encoder + 150 M Nandi), compute is not the bottleneck --- on an H100 the pipeline runs many times faster than real time. The latency a user feels after they stop speaking has historically been dominated not by compute but by the endpoint wait: the silence one observes to be sure the turn is over. We remove that wait architecturally.

The three levers (all in code). (1) **Predictive endpointing** --- the floor-control head is trained with the `<EOS_SPEECH>` label shifted earlier (`eos_lead_ms`, default 160 ms via `events_from_timeline`), so it fires as the turn is finishing rather than after a silence timeout (`endpoint_mode: predictive`). (2) **Small chunks** --- `stream.chunk_ms = 80` sets the reaction granularity. (3) **Zero lookahead** --- the block-causal mask uses `right_chunks: 0` (pure causal), so no future audio is awaited; past context (`left_chunks`) is free because it is already buffered.

Measured budget. `scripts/10_latency_bench.py` reports per-stage p50/p95. The end-of-turn latency is `chunk_ms + compute`:

What it requires, and the honest tradeoff. Two standard streaming-inference pieces must be built (they are not in the smoke-tested code): a cache-aware incremental encoder (each chunk $O(\text{chunk})$, not $O(\text{buffer})$) and KV-cache streaming decode for Nandi (decode tokens as audio arrives, so at end-of-turn only the tail remains). Both are well-trodden; we have a reference (the ThinkSpark live-agent loop). The real tension is not compute but prediction: firing `<EOS_SPEECH>` within ~ 80 ms of the last word means committing before a long silence confirms it, which trades a small false-cut risk for responsiveness. We manage it with the fire threshold ($\tau_{\text{<EOS_SPEECH>}} = 0.55$) and a tiny optional lookahead; a deployment that prefers safety over speed raises the threshold or `right_chunks` to 1. **Claim:** ~ 80 --120 ms

Component	Latency
Endpoint wait (predictive)	≈ 0 (folded into one chunk)
Reaction chunk (<code>chunk_ms</code>)	80 ms
Encoder on the chunk (incremental)	a few ms
Floor-control head (one MLP)	< 1 ms
Nandi finalize (KV-cache streaming: transcript already decoded during speech)	tens of ms
End-of-turn total	$\approx 80\text{--}120$ ms

Table 5: Latency budget with predictive endpointing. The reaction chunk dominates; drop `chunk_ms` to 40 for even lower latency at some accuracy cost. Backchannels fire *during* speech at one chunk (≈ 80 ms).

end-of-turn with predictive endpointing (compute alone < a few tens of ms); backchannels react within one chunk during speech. This is at the frontier --- competitive systems sit at 200--500 ms --- so it is a target backed by the budget above, to be confirmed on the H100 with the latency harness.

Does low latency hurt WER? Mostly no — and here is exactly what does. The latency knobs are deliberately decoupled from transcription accuracy:

- **The 80 ms reaction chunk does NOT affect WER.** It only sets how often we emit a decision. The encoder still attends over its own block (`chunk_frames`= 16, ≈ 320 ms) plus generous past context (`left_chunks`= 4, ≈ 1.28 s). Those --- not the reaction cadence --- determine transcript quality.
- **Predictive endpointing does NOT change what is transcribed.** The encoder and Nandi transcribe the audio regardless of when `<EOS_SPEECH>` fires; endpoint timing only decides when we declare the turn over. A premature EOS is self-healing: the speech-resume guard (`StreamDecider.unlatch`) un-latches and keeps transcribing the moment speech continues, so the tail is never dropped. The only residual risk is the downstream agent replying a touch early --- a turn-taking nicety, not an ASR error.
- **The one real accuracy cost is causal streaming itself** (`right_chunks`= 0, no future context). A purely causal encoder is typically $\sim 1\text{--}3$ absolute WER points worse than the same model run offline (non-causal). Past context is free (no latency), so we use a lot of it; if WER needs the future, `right_chunks`= 1 recovers most of the gap at the cost of one chunk of latency --- an explicit, per-deployment dial.

Consolidated expectations. Table 6 states the numbers we hold ourselves to across the whole design (proposal targets, not results).

15 Key Advancements Aimed For

1. **One system, three tasks.** Transcription, live domain correction, and floor control in a single streaming forward path, replacing a chain of separate models.
2. **Streaming-first validation.** A block-causal encoder mask from Phase 1 means the < 5% WER target is a streaming number, not an offline one that later regresses.
3. **Sub-100 ms end-of-turn.** Predictive endpointing (Section 14) removes the silence wait, targeting $\sim 80\text{--}120$ ms response after the user stops --- roughly $3\text{--}5\times$ faster than typical 200--500 ms pipelines.

Metric	Target	Notes
Offline (non-causal) WER, clean test	5–8%	upper bound of what streaming can approach
Streaming (causal) WER, clean test	< 5% stretch, 6–9% realistic	the main Phase-3 gate; the honest streaming number
Streaming WER, real-world spontaneous	8–13%	≈8% would match the open SOTA (IndicConformer)
Streaming tax (causal vs offline)	+1–3 WER pts	shrinks with <code>right_chunks=1</code>
Floor-control F_1 per signal	> 0.8	with false-fire < 2% on no-flag audio
End-of-turn latency	~80–120 ms	predictive endpoint; compute alone tens of ms
Backchannel reaction (during speech)	≈1 chunk (80 ms)	
Domain-term accuracy (Phase 5)	up, no WER regression	Nandi correction layer

Table 6: Consolidated proposal targets. Biggest lever on WER is *clean data volume*; biggest risk is the Omni SSL encoder (fallback: FastConformer, Section 5).

4. **Stability by construction.** A per-chunk head whose confident default is NOTHING, trained on a large no-flag majority and guarded at inference by threshold and hysteresis, so the agent stays quiet unless a signal is genuinely due.
5. **Audio-aware, distribution-controlled data.** An agent that reads real audio timing and manufactures floor-control supervision with explicit traps, a config-driven scenario mix, and semantic-validity checks; firing is further controllable at inference (per-signal threshold/bias/temperature) so backchannel eagerness is an operator dial.
6. **A reusable pattern.** Fusing a raw SSL encoder with a Hindi-native SLM, extending only a handful of vocabulary tokens, scales to the remaining Indic languages and English without redoing the expensive parts.

References

- [1] Meta AI. Omnilingual ASR: Open-Source Multilingual Speech Recognition for 1600+ Languages. arXiv:2511.09690, 2025.
- [2] FrontiersMind. Nandi-Mini-150M model card. Hugging Face, 2025.
- [3] A. Baevski, H. Zhou, A. Mohamed, M. Auli. wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations. NeurIPS 2020. arXiv:2006.11477.
- [4] A. Graves, S. Fernández, F. Gomez, J. Schmidhuber. Connectionist Temporal Classification. ICML 2006.
- [5] W.-N. Hsu et al. HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units. IEEE/ACM TASLP, 2021. arXiv:2106.07447.
- [6] A. Radford et al. Robust Speech Recognition via Large-Scale Weak Supervision (Whisper). arXiv:2212.04356, 2022.

- [7] Y. Chu et al. Qwen-Audio: Advancing Universal Audio Understanding. arXiv:2311.07919, 2023.
- [8] Y. Chu et al. Qwen2-Audio Technical Report. arXiv:2407.10759, 2024.
- [9] C. Tang et al. SALMONN: Towards Generic Hearing Abilities for Large Language Models. ICLR 2024. arXiv:2310.13289.
- [10] E. Ekstedt, G. Skantze. Voice Activity Projection: Self-Supervised Learning of Turn-Taking Events. Interspeech/SIGDIAL 2022.
- [11] T. Javed et al. Towards Building ASR Systems for the Next Billion Users (IndicWav2Vec). AAAI 2022. arXiv:2111.03945.
- [12] K. Bhogale et al. Effectiveness of Mining Audio and Text Pairs from Public Data (Shrutilipi). arXiv:2208.12666, 2022.
- [13] E. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685, 2021.
- [14] A. Vaswani et al. Attention Is All You Need. NeurIPS 2017.
- [15] Vaani Benchmark V1.0: An Inclusive Multimodal Benchmark Dataset for Hindi. arXiv:2606.21408, 2026.
- [16] Voice of India: A Large-Scale Benchmark for Real-World Speech Recognition in India. arXiv:2604.19151, 2026.
- [17] AI4Bharat. IndicConformer: Multilingual Conformer ASR for 22 Indian Languages. 2024--2026 (ai4bharat/indic-conformer-600m-multilingual).